Circuit Breaker Pattern: An Example

Now that we've delved into the theory and workings of the Circuit Breaker pattern, it's time to bring it all together with a practical Java example. We will illustrate how to integrate a Circuit Breaker into a system, using a simple Online Retail Store as an example.

System Overview: Online Retail Store

Our system is an online retail platform with several microservices:

1. **Product Catalog Service:** Manages product listings.
2. **Order Service:** Handles order placements and management.
3. **Payment Service:** Processes payments.
4. **Shipping Service:** Manages the shipping of items.

We'll focus on the interaction between the Order Service and the Payment Service. The Order Service will use the Circuit Breaker Pattern to communicate with the Payment Service.

Scenario for Circuit Breaker

- The Order Service sends a payment processing request to the Payment Service when a customer places an order.
- If the Payment Service is down or responding slowly, the Order Service should not keep waiting or retrying indefinitely. This is where our Circuit Breaker comes into play.

Components for Coding Example:

1. **Payment Service (Mocked):**

```java
public class PaymentService {
    public String processPayment(Order order) {
        // Simulate payment processing
        // This method can sometimes fail or take a long time to respond
        return "Payment processed for order: " + order.getId();
    }
}
```

2. **Order Class:**

```java
public class Order {
    private String id;
    private double amount;

    // Constructor, getters and setters
}
```

3. **Circuit Breaker Implementation:**
   - We'll use a simple Circuit Breaker class.
   - For a full-fledged system, consider using a library like Resilience4j.

```java
public class CircuitBreaker {
    private CircuitBreakerState state;
    private long lastFailureTime;
    private int failureThreshold;
    private long failureTimeout;

    public CircuitBreaker(int failureThreshold, long failureTimeout) {
        this.state = CircuitBreakerState.CLOSED;
        this.failureThreshold = failureThreshold;
        this.failureTimeout = failureTimeout;
    }

    public synchronized void recordFailure() {
        if (++failures >= failureThreshold) {
            state = CircuitBreakerState.OPEN;
            lastFailureTime = System.currentTimeMillis();
        }
    }

    public synchronized void recordSuccess() {
        state = CircuitBreakerState.CLOSED;
        failures = 0;
    }

    public synchronized boolean canExecute() {
        if (state == CircuitBreakerState.OPEN) {
            if ((System.currentTimeMillis() - lastFailureTime) > failureTimeout) {
                state = CircuitBreakerState.HALF_OPEN;
                return true;
            }
            return false;
        }
        return true;
    }

    // State enum and other methods
}
```

4. **Order Service Using Circuit Breaker:**

```java
public class OrderService {
    private PaymentService paymentService;
    private CircuitBreaker circuitBreaker;

    public OrderService() {
        this.paymentService = new PaymentService();
        this.circuitBreaker = new CircuitBreaker(3, 10000); // e.g., 3 failures, 10-second timeout
    }

    public String placeOrder(Order order) {
        if (circuitBreaker.canExecute()) {
            try {
                String response = paymentService.processPayment(order);
                circuitBreaker.recordSuccess();
                return response;
            } catch (Exception ex) {
                circuitBreaker.recordFailure();
                return handlePaymentServiceDown();
            }
        } else {
            return handlePaymentServiceDown();
        }
    }

    private String handlePaymentServiceDown() {
        // Fallback logic, like queuing the order for later processing
        return "Payment Service is down. Your order is queued.";
    }
}
```

The Pattern in Action:

- When `placeOrder` is called in `OrderService`, it checks with the `CircuitBreaker` if it should proceed.
- If `CircuitBreaker` is in the OPEN state (after too many failures), it uses the fallback logic.
- If in CLOSED or HALF-OPEN state, it tries to process the payment.
- On successful payment, `recordSuccess` is called. On failure, `recordFailure` is called, which might trip the breaker if the threshold is reached.

This example is simplified for illustration. In a real-world application, you'd likely use a library like Resilience4j, which offers advanced features like time-based circuit breaker states, integration with monitoring systems, and more.

← Previous
The Circuit Breaker Pattern: An Effective Shield Against Cascading Failures

Next →
Performance Implications And Special Considerations

☐ Mark as Completed